

## דוח פרויקט רשתות תקשורת מחשבים - חלק 2: מערכת צ'אט

**מגישים:** אביטל יוחננוב, נאור אנידג'ר

בחלק זה של הפרויקט, מימשנו מערכת צ'אט מרובת משתמשים (Multi-Client Chat System) הפועלת על גבי פרוטוקול TCP. המערכת מבוססת על ארכיטקטורת Client-Server, שבה שרת מרכזי מנהל את החיבורים והלקוחות יכולים לשלוח הודעות פרטיות זה לזה.

### ארכיטקטורת הקוד:

- **השרת (server.py):** מאזין בפורט 5000 לכל ממשקי הרשת (0.0.0.0). הוא משתמש ב-Multi-Threading (יצירת Thread נפרד לכל לקוח) כדי לטפל במספר משתמשים במקביל ללא חסימות. הניהול מתבצע באמצעות dictionaries (users, peers) המסונכרנים ע"י מנעול (Lock).
- **הלקוח (gui\_client.py / client.py):** מתחבר לשרת ב-TCP. גם הלקוח פועל בשני threads: אחד לממשק המשתמש (שליחת הודעות) ואחד להאזנה להודעות נכנסות מהשרת בזמן אמת.
- **פרוטוקול:** הגדרנו פרוטוקול טקסטואלי פשוט (MSG, CONNECT, DISCONNECT) המופרד בירידות שורה.

### הוראות התקנה והרצה:

1. יש לוודא שמותקן Python 3.6 ומעלה.
2. **הפעלת השרת:** פתחו טרמינל והריצו: `python server.py`.
3. **הפעלת לקוח:** פתחו טרמינל נפרד והריצו `python gui_client.py` (לממשק גרפי) או `client.py` (לממשק טקסט).
4. יש להזין שם משתמש ולהתחבר.

### דוגמאות קלט ופלט:

להלן דוגמא לתרחיש הרצה של המערכת:

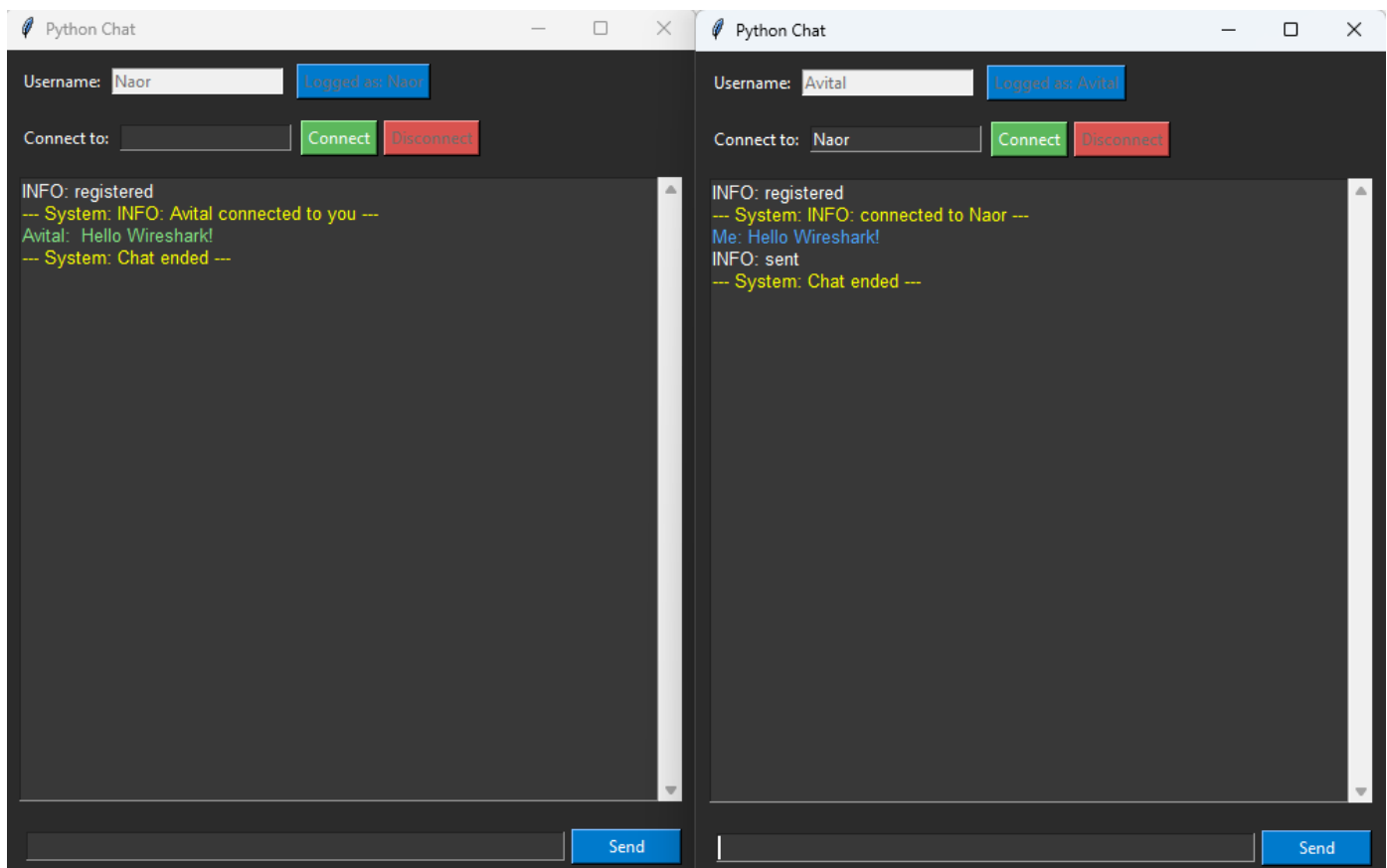
1. מסך השרת (Server Console):

בצילום המסך ניתן לראות את השרת מאזין לפורט ואת הלוגים המעידים על התחברות משתמשים (Avital, Naor) ורישומם במערכת.

```
PS C:\Users\Avital\Desktop\לנס\סידומיל\project part 2> python server.py
Server is running... waiting for clients
Client connected: ('127.0.0.1', 51677)
User registered: Avital
Client connected: ('127.0.0.1', 56996)
User registered: Naor
█
```

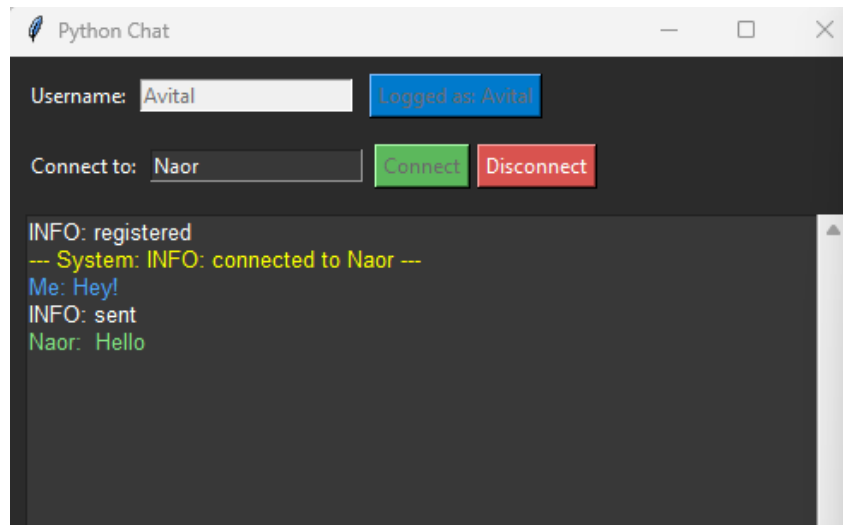
בתמונות הבאות ניתן לראות את ממשק ה-GUI שפיתחנו. המשתמשים מחוברים, ומבצעים את הפקודות הבאות:

- **קלט:** חיבור למשתמש (CONNECT).
- **פלט:** הודעת מערכת "...INFO: connected to".
- **קלט:** שליחת הודעה "Hello Wireshark".
- **פלט:** ההודעה מופיעה בצד אחד ובצד השני בירוק.

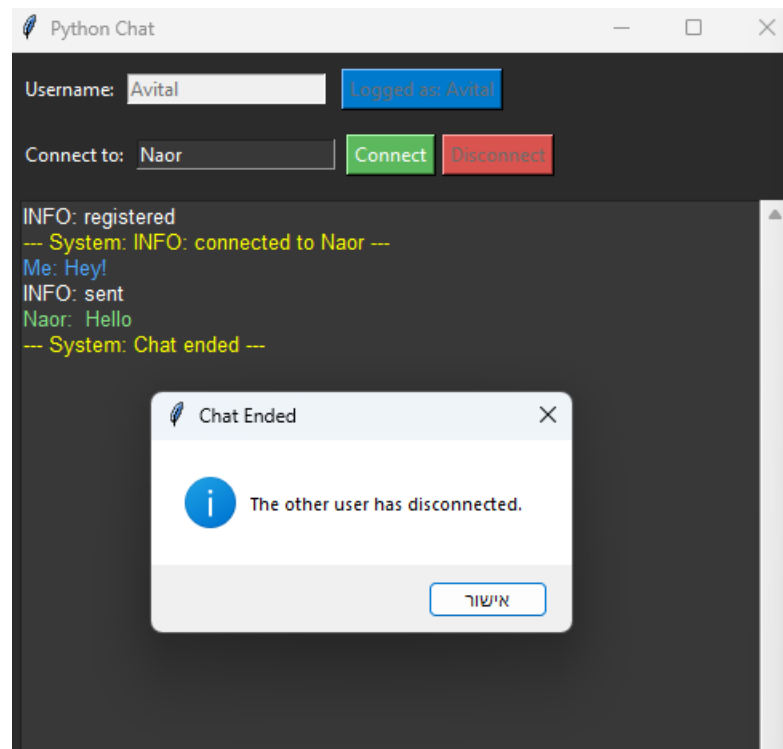


להלן עוד מספר תרחישים במערכת:

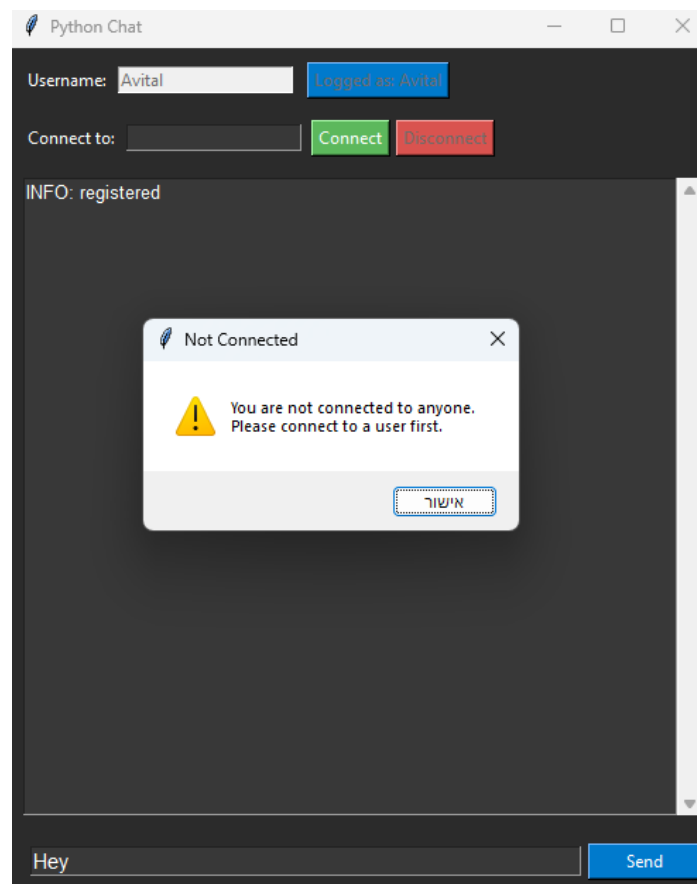
שליחת הודעה תקינה:



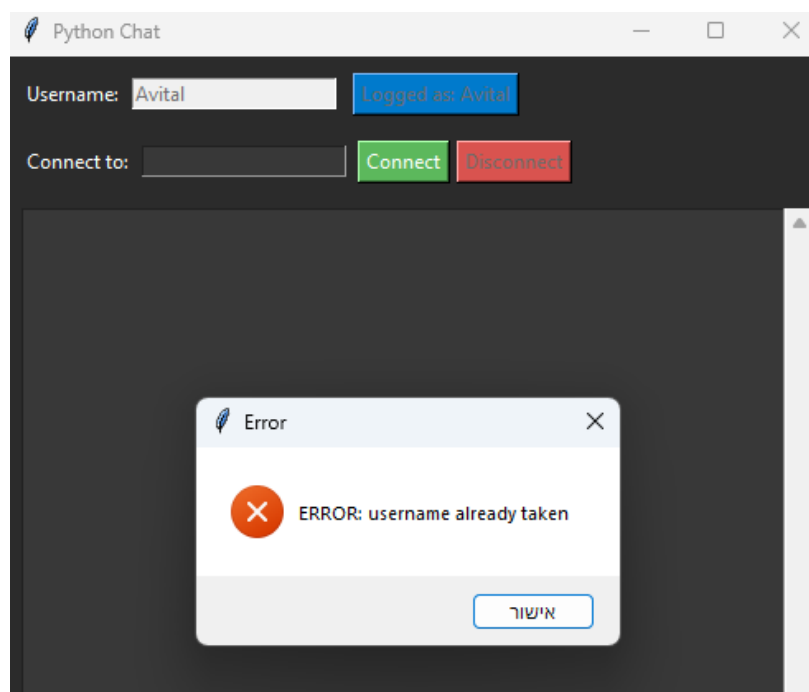
שגיאת "משתמש לא מחובר":



שגיאת "שליחה ללא יעד":



שגיאת "שם משתמש תפוס":



## ניתוח תעבורה של היישום עד שכבת הרשת (Network Layer)

ביצענו לכידת תעבורה ב-Wireshark כדי לוודא שהפרוטוקול פועל כנדרש. הלכידה בוצעה בממשק ה-Loopback.

Adapter for loopback traffic capture

Help Tools Wireless Telephony Statistics Analyze Capture Go View Edit File

+ tcp.stream eq 2

| No. | Time       | Source    | Destination | Protocol | Length | Info                                                                           |
|-----|------------|-----------|-------------|----------|--------|--------------------------------------------------------------------------------|
| 111 | 420.912376 | 127.0.0.1 | 127.0.0.1   | TCP      | 56     | Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM [SYN] 5000 → 51677            |
| 112 | 420.912462 | 127.0.0.1 | 127.0.0.1   | TCP      | 56     | Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM [SYN, ACK] 51677 → 5000 |
| 113 | 420.912502 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=1 Ack=1 Win=65280 Len=0 [ACK] 5000 → 51677                                 |
| 114 | 420.915112 | 127.0.0.1 | 127.0.0.1   | TCP      | 51     | Seq=1 Ack=1 Win=65280 Len=7 [PSH, ACK] 5000 → 51677                            |
| 115 | 420.915145 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=1 Ack=8 Win=65280 Len=0 [ACK] 51677 → 5000                                 |
| 116 | 420.917942 | 127.0.0.1 | 127.0.0.1   | TCP      | 61     | Seq=1 Ack=8 Win=65280 Len=17 [PSH, ACK] 51677 → 5000                           |
| 117 | 420.917998 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=8 Ack=18 Win=65280 Len=0 [ACK] 5000 → 51677                                |
| 118 | 453.219754 | 127.0.0.1 | 127.0.0.1   | TCP      | 57     | Seq=8 Ack=18 Win=65280 Len=13 [PSH, ACK] 5000 → 51677                          |
| 119 | 453.219794 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=18 Ack=21 Win=65280 Len=0 [ACK] 51677 → 5000                               |
| 120 | 453.220064 | 127.0.0.1 | 127.0.0.1   | TCP      | 68     | Seq=18 Ack=21 Win=65280 Len=24 [PSH, ACK] 51677 → 5000                         |
| 121 | 453.220094 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=21 Ack=42 Win=65280 Len=0 [ACK] 5000 → 51677                               |
| 122 | 453.220094 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=21 Ack=42 Win=65280 Len=21 [PSH, ACK] 5000 → 51677                         |
| 123 | 464.219510 | 127.0.0.1 | 127.0.0.1   | TCP      | 65     | Seq=21 Ack=42 Win=65280 Len=21 [PSH, ACK] 5000 → 51677                         |
| 124 | 464.219548 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=42 Ack=42 Win=65280 Len=0 [ACK] 51677 → 5000                               |
| 125 | 464.219798 | 127.0.0.1 | 127.0.0.1   | TCP      | 55     | Seq=42 Ack=42 Win=65280 Len=11 [PSH, ACK] 51677 → 5000                         |
| 126 | 464.219817 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=42 Ack=53 Win=65280 Len=0 [ACK] 5000 → 51677                               |
| 127 | 467.187298 | 127.0.0.1 | 127.0.0.1   | TCP      | 55     | Seq=42 Ack=53 Win=65280 Len=11 [PSH, ACK] 5000 → 51677                         |
| 128 | 467.187337 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=53 Ack=53 Win=65280 Len=0 [ACK] 51677 → 5000                               |
| 129 | 467.187558 | 127.0.0.1 | 127.0.0.1   | TCP      | 73     | Seq=53 Ack=53 Win=65280 Len=29 [PSH, ACK] 51677 → 5000                         |
| 130 | 467.187586 | 127.0.0.1 | 127.0.0.1   | TCP      | 44     | Seq=53 Ack=82 Win=65280 Len=0 [ACK] 5000 → 51677                               |

0000 02 00 00 00 45 00 00 35 bb 5f 40 00 00 06 00 00 .....E..5 ..\_@..... ket, 57 bytes on wire (456 bits), 57 bytes captured (456 bits) on interface \Device\NPF\_{Loopback, id 0 <  
0010 7f 00 00 01 7f 00 00 01 c9 dd 13 88 6b bd 77 f2 .....k.w..... Null/Loopback <  
0020 e2 dd bb c0 50 18 00 ff 5e 35 00 00 43 4f 4e 4e .....P....^5..CONN Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 <  
0030 45 43 54 20 4e 61 6f 72 0a ECT Naor > Transmission Control Protocol, Src Port: 51677, Dst Port: 5000, Seq: 8, Ack: 18, Len: 13 <  
Data (13 bytes) <

Profile: Default | Packets: 148 · Displayed: 19 (12.8%) · Dropped: 0 (0.0%) | wireshark\_Adapter for loopback traffic capture9ZCPI3.pcapng

Wireshark · Follow TCP Stream (tcp.stream eq 2) · Adapter for loopback traffic capture

Avital

INFO: registered

CONNECT Naor

INFO: connected to Naor

MSG Hello Wireshark!

INFO: sent

DISCONNECT

INFO: disconnected from Naor

## ניתוח השכבות (לפי התעבורה שנלכדה):

1. **שכבת האפליקציה:** ניתן לראות בבירור את תוכן הפרוטוקול שלנו כטקסט קריא (Plaintext): הפקודות Naor, CONNECT, MSG Hello Wireshark! עוברות בדיוק כפי שהוגדרו בקוד.
2. **שכבת התעבורה (TCP - Transport):** המידע מועבר על גבי פרוטוקול TCP. בצילום ניתן לראות את ה-Stream המלא, ואת הדגלים ACK, PSH, המעידים על דחיפת המידע ליישום. פורט היעד הוא 5000.
3. **שכבת הרשת (IP - Network):**
  - **כתובות IP:** מכיוון שההרצה הייתה מקומית, כתובת המקור (Source IP) וכתובת היעד (Destination IP) הן **127.0.0.1** (Loopback).
  - **פרוטוקול IP:** בשכבה זו, הפרוטוקול המזוהה הוא IPv4, והוא נושא בתוכו את סגמנט ה-TCP.
  - ניתוח זה מוכיח שהמידע עבר את כל תהליך ה-Encapsulation משכבת האפליקציה ועד לרמת ה-IP בצורה תקינה.

## תיאור שימוש בבינה מלאכותית (AI)

בפרויקט זה נעזרנו בכלי AI למספר מטרות:

1. **ייעול האלגוריתמיקה:** עזרה בתכנון נכון של מנגנון ה-Multi-threading בשרת למניעת התנגשויות.
2. **שיפור ממשק המשתמש (GUI):** סיוע בכתיבת קוד ה-Tkinter ליצירת עיצוב מודרני (Dark Mode) ונוח.

## דוגמאות לפרומפטים:

- "כיצד ניתן לטפל במספר לקוחות במקביל ב-Python באמצעות שימוש ב-sockets ו-threading מבלי לחסום את הריצה (non-blocking)?"
- "צור קוד לממשק גרפי (GUI) במצב כהה (Dark Mode) עבור אפליקציית צ'אט ב-Tkinter, עם צבעים נפרדים להודעות שנשלחו ולהודעות שהתקבלו."